

Slide 1

Toegepaste Informatica

2024

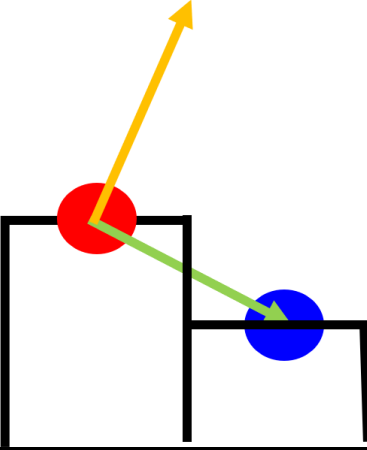
Week 7: Aliasing



Last class

- Textures
- Normal maps

EXAMPLE



The diagram shows a 2D wireframe of a staircase with two steps. A red sphere is positioned on the top step, and a blue sphere is on the bottom step. An orange arrow originates from the red sphere and points upwards and to the right, representing an incident ray. A green arrow originates from the same point on the red sphere and points downwards and to the right, hitting the blue sphere, representing a reflected ray. To the left of the staircase is a blue rectangular area containing the word 'EXAMPLE' in a stylized, multi-colored font, with four horizontal lines above and below it. To the right of the staircase is a small 3D inset image showing a character standing on a textured ground plane with a white path.

Course schedule

Highly likely to change ☹



		Wed
Week 1		Introduction
Week 2		3D reasoning
Week 3		Geometry
Week 4		Pipeline & Materials
Week 5		Proceduralism
Week 6		Textures & Normal maps
Week 7		Aliasing / Presentations
Week 8	Advanced pipeline: Forward rendering	Advanced pipeline: Deferred rendering
Week 9	?	Post Processing
Week 10	?	Project
Week 11	?	Revision

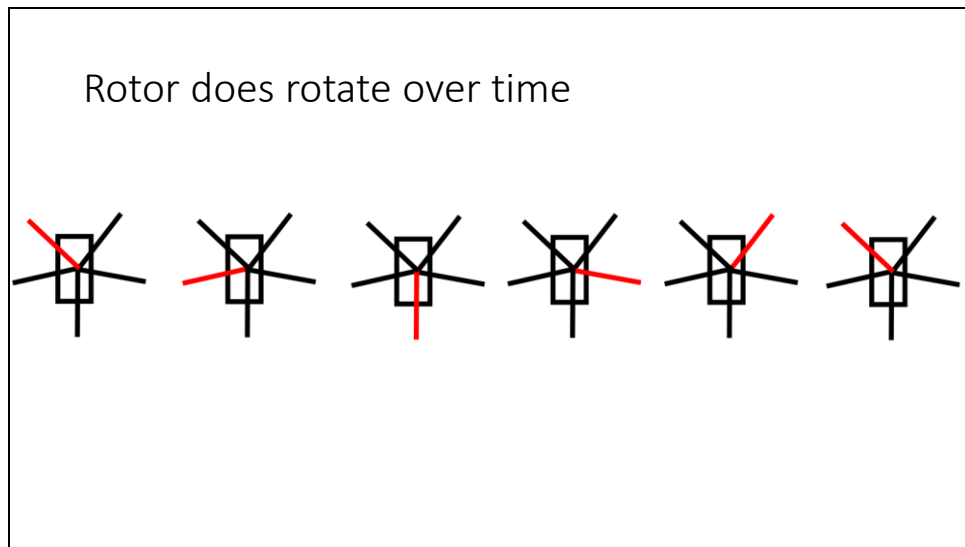
This class

- → Temporal aliasing←
- Aliasing
- Anti – Aliasing

Temporal aliasing

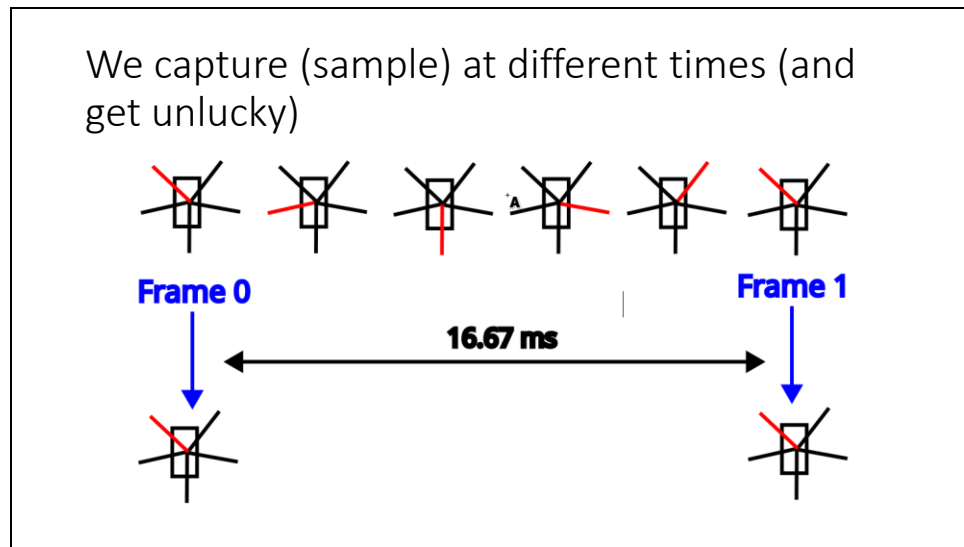


The helicopter obviously has his rotor (the part that moves at the top) moving. But why does it appear the rotor stays still? Think about it for a second before moving on to the next slide.



I highlighted one of the blades in red, to make it easier to see what's going on

Above shows the rotor turning, as it would when we observe it directly (not on video)



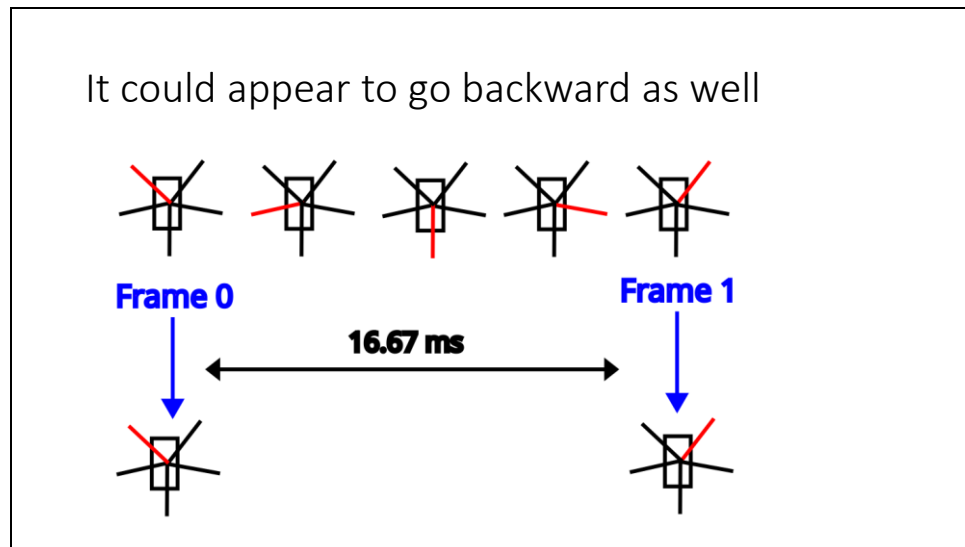
Above, we still have the helicopter as we look at it in real life

Beneath, I've visualized what a camera would see. A camera works like a display, capturing tens of images every second

We see that it captures the same image twice, due to the time for the rotor to rotate 360 degrees being the same as the time between two frames. In a movie, it would look like the helicopter doesn't move at all

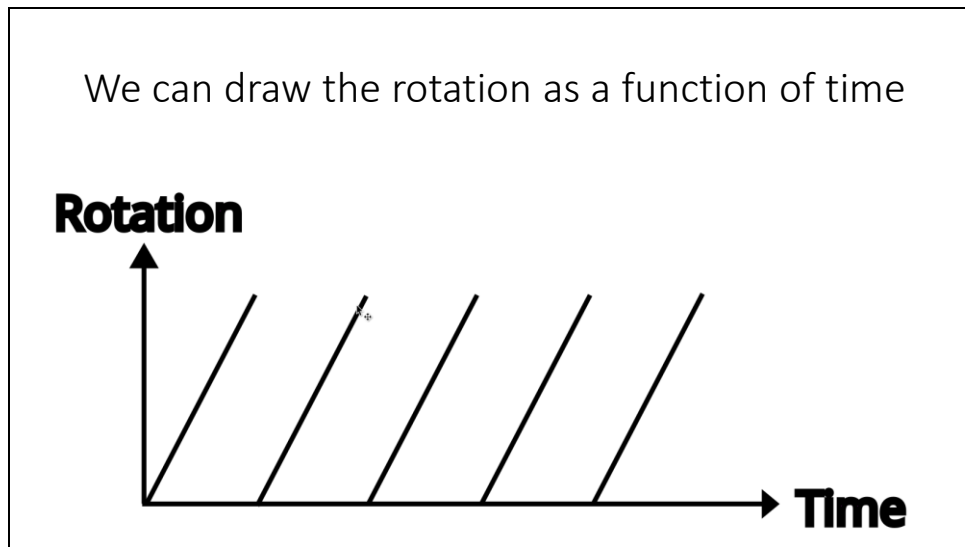
Aliasing put very, very simply is when we "show something different than what it really is"

In this case, we're showing a helicopter with a rotor that doesn't move, while in reality, the rotor is moving very fast.

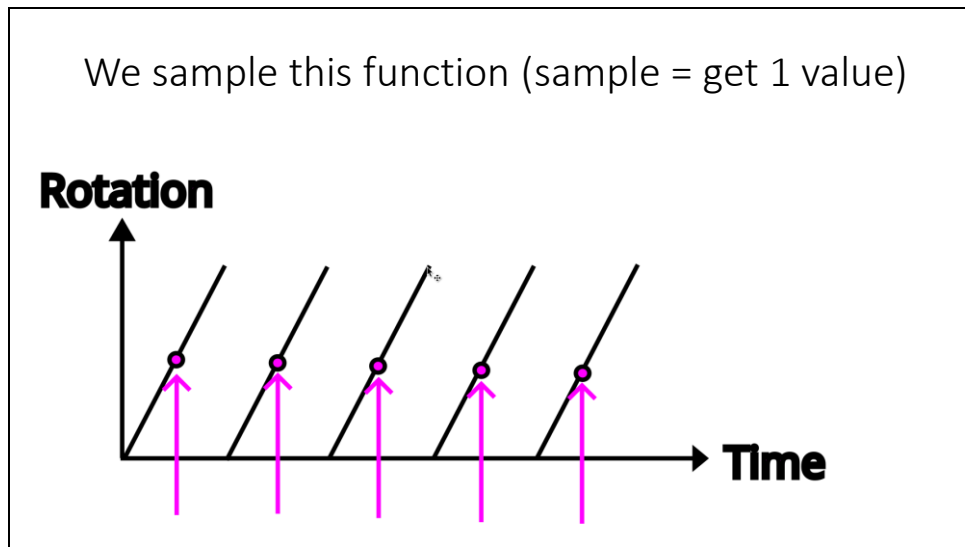


Another example

Here, in reality, the rotor is turning counterclockwise
But for the camera, it's turning clockwise

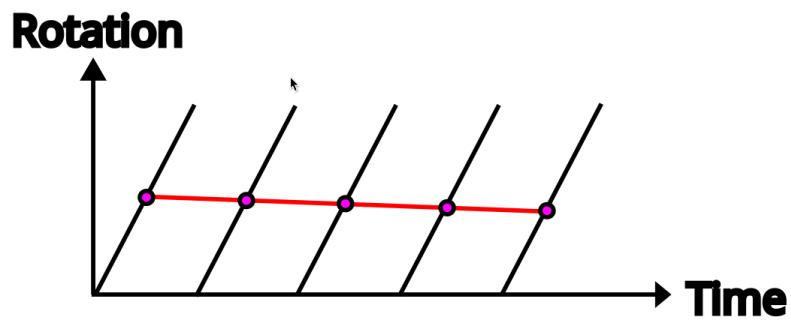


This shows the angle of that single blade over time, going from 0 to 360 degrees. It's choppy as it jumps from 360 $\rightarrow$ 0



Here in magenta, the arrows are the times that the camera captures the rotor, and the dots are the rotation the rotor is at when the camera takes a picture

And get a new function (a reconstruction)

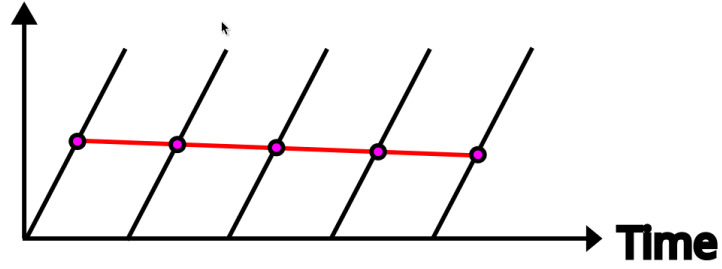


Red it what the rotation over time looks like for the camera. Again, almost no rotation, even though the actual rotation (in black) happens fast

And it's totally wrong!

- We don't take enough samples
- We get a function that is aliasing the actual function

Rotation



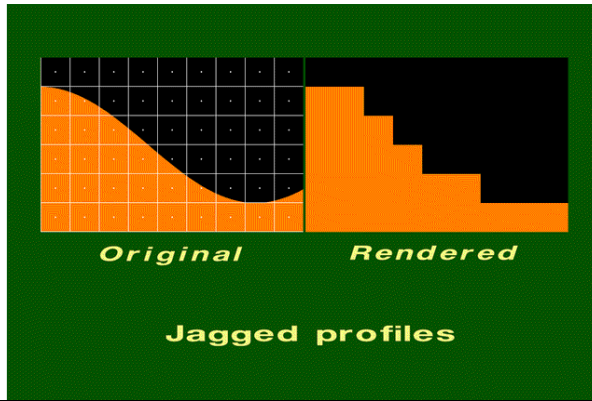
This class

- Temporal aliasing
- → Aliasing ←
- Anti – Aliasing

TODO show foveation deck

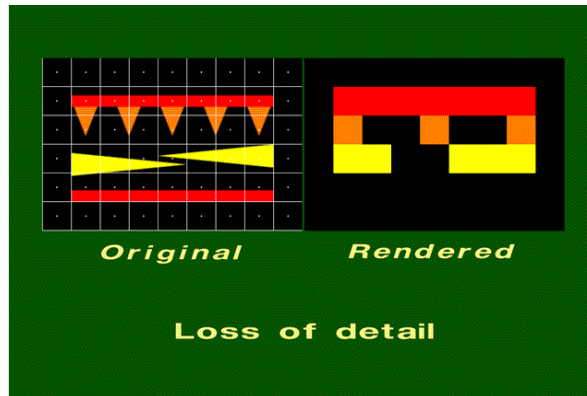
There's another powerpoint this class, that ends in _fr. Check that one now, and come back here when you're done

Example of staircasing



An example how the typical jagged staircase happens. The curve is too complicated for the resolution

Too much geometric detail



Same here. If something covers the center of a pixels, it turns into a square pixel of that color. But the results looks nothing like the original. The triangles are too small

Too much texture detail



Here, in the front, we have enough pixels to show the checkered pattern. In the distance, we suddenly start skipping checks, and the whole pattern breaks apart.

This class

- Temporal aliasing
- Aliasing
- → Anti – Aliasing ←

Anti - Aliasing

- We can never remove aliasing completely without sampling more
- Why?
- We can avoid the problem
- We can remove the resulting issues

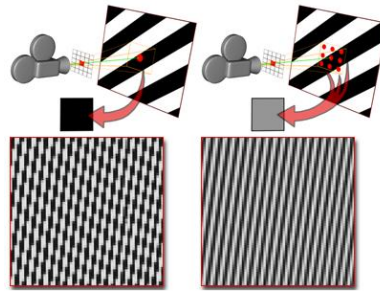
Anti - Aliasing

Varying techniques that help our renderer alias less
Some can coexist (LODing is almost always done)

- Supersampling
- MSAA
- FXAA
- TAA
- LOD's

Supersampling

- Render at a higher resolution
- Downsample to display resolution
- Eg display is 1920 x 1080 → render at 3840 x 2160 (4 samples per pixels)
- Works very well
- Very expensive and wasteful (most pixels don't need more samples)
- Pixar does this (thousands of samples per pixel)



MSAA

- Multi sample antialiasing
- More samples at the edge of objects
- Works well with staircasing
- “more samples where we need them”
- Fragment shader run for each object!
- Doesn't work with modern pipelines

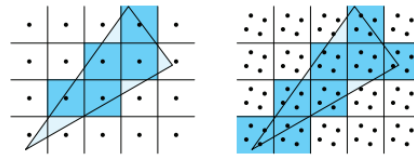


Figure 23.4. The effect of multisampling on fragment shader load. Without multisampling, only pixels whose centers are covered generate fragments (left; blue). If any of the sample points are covered, a fragment is generated, thus leading to slightly more fragments along edges (right).

FXAA

- Full screen pass (post process)
- Doesn't avoid aliasing
- Tries to hide it
- "edge blurrer" : how does blur work?
- Easy and cheap



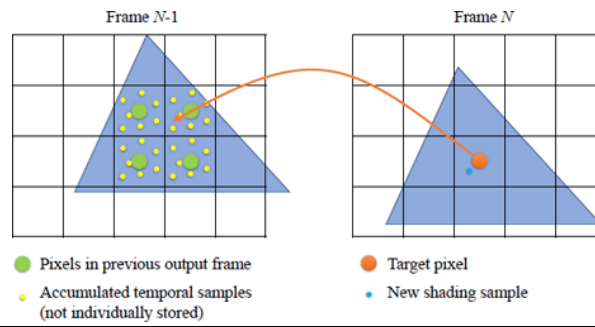
TAA

- Temporal Anti Aliasing
- Gather samples over different frames
- Complex : camera moves (have to find previous position of pixel)
- Preferred method for AAA games



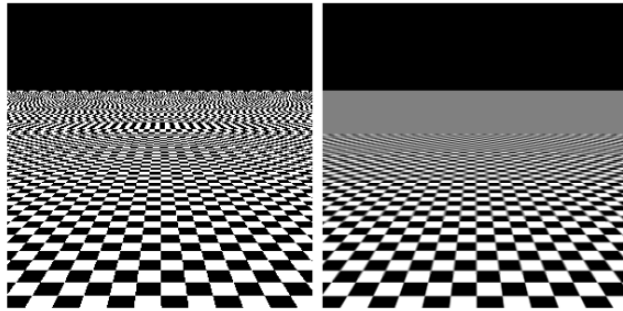
Don't look too long at the image, it's a bad one. Just remember TAA works by looking at multiple frames

- Every frame:
Take exact position on triangle: calculate in previous frame
Take previous samples to detect aliasing
Eg 3 blue samples, one white : should be blue



LOD's

- Use less detail in the distance, where there are less samples
- Which one looks better?
- Which one is more correct?

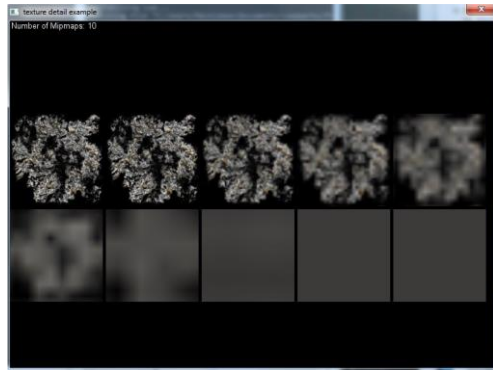


I hope you agree the right looks better

The right one is more correct because it more accurately represents the checkers: gray in the distance, because the squares blend together (white and black blended is gray). The left shows a weird, wavy checkers that is wrong.

Mipmaps

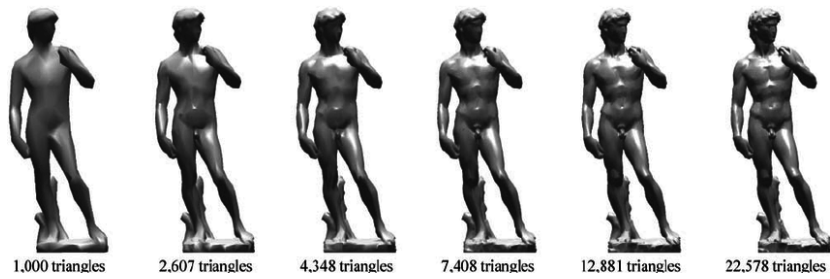
- Mipmaps always have a dimensions / 2 compared to the previous mip
- It's not a "simpler" version, it's correctly filtered (less small detail)



Each texture has an array of smaller textures, aka 'mips'. When the texture is being far away (with too much detail), it uses one of the smaller (blurrier) mips

Mesh LOD's

- Big difference with textures: the entire model is one LOD, while textures are sampled across all the LOD's



We also do this with meshes, but it's simpler. We have a bunch of models with different triangle counts. When rendering, we choose the most appropriate one based on the distance between the model and the camera

Recap

- Aliasing happens when you have too much detail (geometry, texture, shader) for your resolution
- Looks like creeping pixels, loss of structure
- Common on very thin geometry: grass, telephone wire, hair
- Supersampling, fxaa, taa can help
- LODing is very important

Next class(es)

- Advanced pipeline concepts
- High level algorithms for using the GPU
- Accounting for light, shadow, and camera's

